Efficient Verification of ω-Regular Properties via Proof Certificates

With the increasing concern for system safety in recent years, dynamical systems are now widely used for modeling in control systems. However, the verification of dynamical systems is challenging, specifically in determining whether the behaviors of these systems are consistent with the specifications. A closure certificate, defined over the state pairs of a dynamical system, is a real-valued function with non-increasing and non-negative properties that prevents reaching a specific state space infinitely many times. When combined with decision tools like sum-of-squares(SOS) and satisfiability-modulo-theory(SMT) solvers, closure certificates can ensure persistence through automated deductive verification methods. Closure certificates can be applied to product systems to verify LTL and ω-regular properties. They achieve this by describing transition invariants from initial states, ensuring that accepting states in ω-automata are visited only finitely many times, thereby blocking all accepting runs. This is not unexpected, because closure certificates are defined on state pairs, leading to a much larger search space compared to the state-triplet method that uses barrier certificates, which reduces synthesis efficiency. This paper proposes several proof certificates as a compromise method inspired by both closure certificates and the state-triplet method to improve synthesis efficiency. We validate these definitions using SOS and SMT-based methods for certificate synthesis, and showcase their effectiveness through case studies.

ACM Reference Format:
. 2018. Efficient Verification of ω-Regular Properties via Proof Certificates. *Proc. ACM Meas. Anal. Comput. Syst.* 37, 4, Article 111 (August 2018), 18 pages. https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

Discrete-time dynamical systems are important mathematical models used to describe the characteristics of state changes in control systems. While these models can effectively capture system behavior, their infinite state space presents challenges for verification. Current methods for verifying temporal properties of these systems include the state-triplet method [31] and closure certificates [19]. However, these methods introduce new concerns: (i) The state-triplet method suffers from conservatism; (ii) The closure certificates have low synthesis efficiency. Consequently, developing a novel, more efficient approach that reduces conservativeness for verifying temporal properties in these systems remains a significant challenge.

Prior research has focused on formal verification of temporal properties—including safety, reachability, and reach-avoid specifications—within dynamical systems [3, 20, 32, 34, 35]. However, specifications for complex systems inherently involve intricate temporal behaviors better captured by Linear Temporal Logic (LTL) [22] and broader ω-regular languages [29]. As detailed below, verifying LTL and ω-regular properties in control systems introduces substantial challenges. First, these properties describe behaviors across infinite execution trajectories [4], typically modeled via ω-automata. When employing proof certificates for verification, the state-triplet method requires unrolling all simple paths and blocking each individually. It is non-trivial to determine the number of certificates needed for synthesis and this method is conservative. Alternatively, closure certificates face the issue of enormous search spaces, as certificates are defined over state pairs. Finally, the verification problem is

Author's Contact Information:

ACM 2476-1249/2018/8-ART111
https://doi.org/XXXXXXX.XXXXXXX

further complicated by the infinite state space and nonlinear dynamic characteristics of control systems, which collectively impose major obstacles for traditional verification methods.

In this work, we consider the formal verification of ω-regular properties for discrete-time dynamical systems using proof certificates. As an abstraction-free method, proof certificates have gained popularity in verification and static analysis domains[6–9]. Murali et al.[19] proposed closure certificates, proof certificates defined as real-valued functions over state pairs of dynamical systems, for persistence verification, which can be used to validate systems against ω-regular properties. Wongpiromsarn et al.[31] introduced the state-triplet method, which demonstrates that a system satisfies ω-regular properties by blocking all reachable paths. Inspired by these two methods, we propose state safety certificates and transition persistence certificates. Their combined use enables verification of general ω-regular specifications expressed via Nondeterministic Büchi Automata (NBA)[5]. Based on these certificates, we present automated synthesis methods using Satisfiability Modulo Theories (SMT) and Sum-of-Squares (SOS) optimization. We theoretically prove that state persistence certificates exhibit stronger expressive power than closure certificates. Furthermore, we demonstrate that their synergistic application alleviates the conservatism of state-triplet methods while reducing search spaces compared to closure certificates. The effectiveness of our synthesis approaches are empirically validated through two case studies.

The organization of this paper is as follows: Section 2 covers preliminary knowledge, including system definitions, specification descriptions, verification methods, and an introduction to some certificates. Section 3 introduces proof certificates and automata-based methods, we prove the validity of proposed certificates, and utilize product system to validate ω-regular properties. Section 4 discusses comparisons with other methods, including the state-triplet method and the closure certificate. Section 5 presents the synthesis methods, including those based on SMT and SOS. Section 6 introduces case studies that demonstrate the effectiveness and efficiency of the synthesis methods. Section 7 summarizes the paper and outlines future work.

Related works: Verification of temporal logic properties via proof certificates has seen significant advancements. To validate discrete-time systems against ω-regular specifications, Wongpiromsarn et al.[31] developed a state-triplet method that conservatively identifies barrier certificates [25] to obstruct transitions between automaton edges, thereby preventing the system from entering accepting states. Murali et al.[19] introduced closure certificates for persistence verification, which establish disjunctively well-founded transition invariants [23] for dynamical systems. Abate et al.[1] adopted Street's supermartingale theory to qualitatively analyze ω-regular properties in discrete-time dynamical systems. Additionally, substantial research has addressed the verification of specific temporal behaviors such as reachability, safety, and avoidance[6–9]. Converting LTL specifications into Transition-based Generalized Büchi Automata (TGBA) typically yields smaller automata compared to Labeled Generalized Büchi Automata (LGBA) [10]. The degeneralization process [15] can convert a TGBA with multiple acceptance conditions into a Transition-based Büchi Automaton (TBA) featuring a single acceptance condition. Various tools exist for generating different automaton types from LTL formulas, such as Owl [18] and SPOT [12].

2 Preliminaries

2.1 Notation

We use $\mathbb{N}$ and $\mathbb{R}$ to represent the set of natural numbers and the set of real numbers, respectively. We define:

$$\mathbb{N}_{\sim i} = \{n \in \mathbb{N} \mid n \sim i\}, \quad \text{where } i \in \mathbb{N}, \sim \in \{\geq, \leq, >, <\},$$
$$\mathbb{R}_{\sim i} = \{r \in \mathbb{R} \mid r \sim i\}, \quad \text{where } i \in \mathbb{R}, \sim \in \{\geq, \leq, >, <\}.$$

Given a set S and a set $S_1 \subseteq S$, we define an indicator function:

$$I_S(x) = \begin{cases} 1, & \text{if } x \in S \\ 0, & \text{if } x \notin S \end{cases}$$

$|S|$ denotes the cardinality of S and $S \backslash S_1$ denotes $\{x \in S \mid x \notin S_1\}$.
Given a sequence $s = \langle s_0, s_1, \dots \rangle$, where $s_i \in S$ for $i \in \mathbb{N}$.

- $s[i]$ denotes s_i;
- $s[i : j]$ denotes $\langle s_i, \dots, s_j \rangle$ for $j \geq i$;
- $s[i : \infty]$ denotes $\langle s_i, s_{i+1}, \dots \rangle$.

We say s visits S if there exists $i \in \mathbb{N}$ such that $s_i \in S$. $Inf(s)$ denotes the set of the elements that appear in s infinitely often. S^ω denotes $\{s \mid s = \langle s_0, s_1, \dots \rangle$ is an infinite sequence and $s_i \in S$ for $i \in \mathbb{N}\}$. A function $f : S \to \mathbb{R}$ is bounded if and only if there exists $a, b \in \mathbb{R}$ such that $a \leq f(x) \leq b$ for all $x \in S$.

2.2 Discrete-time Dynamical System

A discrete-time dynamical system $\mathfrak{S}$ is a tuple $(\mathcal{X}, \mathcal{X}_0, f)$, where $\mathcal{X} \subseteq \mathbb{R}^n$ represents system states, $\mathcal{X}_0 \subseteq \mathcal{X}$ represents initial states, and $f \subseteq \mathcal{X} \times \mathcal{X}$ represents the transition relation. The evolution of the system state is described as follows:

$$\mathfrak{S} : x_{t+1} = f(x_t)$$

The state space is compact. A infinite sequence of states $\langle x_0, x_1, x_2, \dots \rangle$ is called a *trajectory* of the system if $x_0 \in \mathcal{X}_0$ and $x_{i+1} \in f(x_i)$ for $i \in \mathbb{N}$. We use $\mathfrak{T}(\mathfrak{S})$ to represent all *trajectories* on $\mathfrak{S}$. The *predecessor* of x_{i+1} is x_i for $i \in \mathbb{N}$. In a given *trajectory*, x_{i+1} has only one *predecessor*. We use the notation x_{i+1}^{pre} to denote the *predecessor* of x_{i+1} in a *trajectory*.

2.3 Specification

Safety, persistence, LTL, and ω-regular properties are interested in this paper. For these specifications, we propose certificates to demonstrate that the discrete-time dynamical system satisfies the specifications.

Safety: A system $\mathfrak{S} = (\mathcal{X}, \mathcal{X}_0, f)$ is *safe* with respect to a set of unsafe states $\mathcal{X}_u \subseteq \mathcal{X}$ if and only if for any *trajectory* $\tau = \langle x_0, x_1, \dots \rangle \in \mathfrak{T}(\mathfrak{S})$, we have $x_i \notin \mathcal{X}_u$ for $i \in \mathbb{N}$.

Persistence: A system $\mathfrak{S} = (\mathcal{X}, \mathcal{X}_0, f)$ is *persistent* with respect to a set of states $\mathcal{X}_{VF} \subseteq \mathcal{X}$ if and only if for any *trajectory* $\tau = \langle x_0, x_1, \dots \rangle \in \mathfrak{T}(\mathfrak{S})$, elements in x_{VF} appear finitely often in τ.

Linear Temporal Logic: Formulae in LTL [22] are defined with respect to a set of atomic propositions AP. AP is used to describe properties of system states. A subset of AP is a *letter*, and the set of all subsets of AP is called the *alphabet*, denoted as Σ. Given a system $\mathfrak{S} = (\mathcal{X}, \mathcal{X}_0, f)$, a labelling function $\mathfrak{L} : \mathcal{X} \to \Sigma$ maps system states to corresponding *letters*. Given a *trajectory* $\tau = \langle x_0, x_1, \dots \rangle \in \mathfrak{T}(\mathfrak{S})$, we can use $\mathfrak{L}$ to mark the propositions from AP for each x_i. We refer to $w_i = \mathfrak{L}(x_i)$ as the *letter* corresponding to the system state x_i. Then, we use the notation $\mathfrak{L}(\tau)$ to represent $\langle \mathfrak{L}(x_0), \mathfrak{L}(x_1), \dots \rangle$, which denotes the infinite sequence of *letters* corresponding to all states in τ.

The syntax of LTL is defined inductively as follows:

$$\phi := \top \mid a \mid \neg\phi \mid \phi \wedge \phi \mid X\phi \mid \phi U \phi$$

where $\top$ indicates true, $a \in AP$ denotes an atomic proposition. $\wedge$ and $\neg$ can be used to derive boolean operators such as or ($\vee$), implication ($\Rightarrow$). X and U can be used to derive temporal operators such as finally (F), always (G).

Given a *word* $w = \mathfrak{L}(\tau)$ and a LTL formula ϕ, the semantics of ϕ is defined inductively as follows:

- $w \models \top$ always holds
- $w \models a$ iff $a \in w[0]$

- $w \models \phi_1 \wedge \phi_2$ iff $w \models \phi_1$ and $w \models \phi_2$
- $w \models \neg\phi_1$ iff $w \not\models \phi_1$
- $w \models X\phi_1$ iff $w[1:\infty] \models \phi_1$
- $w \models \phi_1 U \phi_2$ iff there exists $j \in \mathbb{N}$ such that $w[j:\infty] \models \phi_2$ and for all $i \in \mathbb{N}_{<j}$, $w[i:\infty] \models \phi_1$

We use $\mathfrak{S} \models_{\mathfrak{L}} \phi$ to indicate that for any $\tau \in \mathfrak{T}(\mathfrak{S})$, $\mathfrak{L}(\tau) \models \phi$.

Nondeterministic Büchi Automaton: A Nondeterministic Büchi Automaton (NBA) is a tuple $(\Sigma, Q, q_0, \delta, Acc)$, where:

- Σ is the *alphabet*,
- Q is a finite set of states,
- $q_0 \in Q$ is an initial state,
- $\delta \subseteq Q \times \Sigma \times Q$ is the transition relation, and
- Acc is the acceptance condition.

Given a transition tuple $t = (s, \sigma, e) \in \delta$, we define s as the start state of t. For any word $w \in \Sigma^\omega$, the run descriptions on w can be categorized into two types:

(1) **State-based run**: $r_{\text{state}} = \langle r_0, r_1, \ldots \rangle$, where $r_0 = q_0$ and $(r_i, w_i, r_{i+1}) \in \delta$ for all $i \in \mathbb{N}$.
(2) **Transition-based run**: $r_{\text{transition}} = \langle (r_0, w_0, r_1), (r_1, w_1, r_2), \ldots \rangle$, where $r_0 = q_0$ and $(r_i, w_i, r_{i+1}) \in \delta$ for all $i \in \mathbb{N}$.

This paper discusses two types of NBA: *state-based NBA* and *transition-based NBA*. The primary distinction lies in their accepting conditions:

- For **the state-based accepting condition**, it is defined by a state set $Acc_{\text{state}} \subseteq Q$. These states are referred to as **accepting states**. A state-based NBA $\mathcal{A}$ accepts a word w if and only if there exists a run r_{state} on w such that states in Acc_{state} occur infinitely often, i.e., $Inf(r_{\text{state}}) \cap Acc_{\text{state}} \neq \emptyset$.
- For **the transition-based accepting condition**, it is defined by a transition set $Acc_{\text{transition}} \subseteq \delta$. These transitions are referred to as **accepting transitions**. A transition-based NBA $\mathcal{A}$ accepts a word w if and only if there exists a run $r_{\text{transition}}$ on w such that transitions in $Acc_{\text{transition}}$ occur infinitely often, i.e., $Inf(r_{\text{transition}}) \cap Acc_{\text{transition}} \neq \emptyset$.

Based on different types of NBA, we define the following sets for following automata-based methods(3):

$$Acc^{pair}_{state} = \{(p, q) \mid p \in Q \wedge q \in Acc_{state} \wedge \exists \sigma \in \Sigma.(p, \sigma, q) \in \delta\}, \tag{1}$$

$$Acc^{start}_{state} = Acc_{state}, \tag{2}$$

$$Acc^{k,l}_{state} = \{(q_k, \sigma, q_l) \mid (q_k, \sigma, q_l) \in \delta \wedge (q_k, q_l) \in Acc^{pair}_{state}\}, \tag{3}$$

$$Acc^{pair}_{transition} = \{(p, q) \mid \exists \sigma \in \Sigma.(p, \sigma, q) \in Acc_{transition}\}, \tag{4}$$

$$Acc^{start}_{transition} = \{p \mid (p, \sigma, q) \in Acc_{transition}\}, \tag{5}$$

$$Acc^{k,l}_{transition} = \{(q_k, \sigma, q_l) \mid (q_k, \sigma, q_l) \in Acc_{transition}\}, \tag{6}$$

- $Acc^{k,l}_{\text{state}}$: The set of transitions that can reach the accepting state q_l from q_k.
- $Acc^{\text{start}}_{\text{state}}$: The set of accepting states.
- $Acc^{k,l}_{\text{transition}}$: The set of accepting transitions from q_k to q_l.
- $Acc^{\text{start}}_{\text{transition}}$: The set of starting states of accepting transitions.

We denote $\mathfrak{L}(\mathfrak{S})$ as all labelled *trajectories*, i.e., $\mathfrak{L}(\mathfrak{S}) = \{\mathfrak{L}(\tau) \mid \tau \in \mathfrak{T}(\mathfrak{S})\}$. This set is referred to as the *language* of $\mathfrak{S}$. The set of *words* accepted by an NBA $\mathcal{A}$ is denoted as $\mathfrak{L}(\mathcal{A})$, which is called the *language* of $\mathcal{A}$. The set of *words* satisfying an LTL formula ϕ is called the *language* of ϕ and is denoted as $\mathfrak{L}(\phi)$. NBAs can express ω-regular and LTL specifications [30]. We adopt the *transition-based* NBA to represent LTL specifications

and the *state-based* NBA to describe ω-regular specifications in this paper. Since specifications are expressed via automata, we can reduce the verification problem of $\mathfrak{S}$ to a *language* inclusion problem. Given a system $\mathfrak{S} = (\mathcal{X}, \mathcal{X}_0, f)$ and an NBA $\mathcal{A} = (2^{AP}, Q, q_0, \delta, Acc)$ to express the specification, we say $\mathfrak{S}$ satisfies $\mathcal{A}$ under the labelling function $\mathfrak{L}(x) : \mathcal{X} \to 2^{AP}$ if and only if for any *trajectory* τ in $\mathfrak{T}(\mathfrak{S})$, $\mathcal{A}$ accepts the *word* $\mathfrak{L}(\tau)$, i.e. $\mathfrak{L}(\mathfrak{S}) \subseteq \mathfrak{L}(\mathcal{A})$. We denote $\mathfrak{S}$ satisfies $\mathcal{A}$ under $\mathfrak{L}$ as $\mathfrak{S} \models_{\mathfrak{L}} \mathcal{A}$.

2.4 Certificates

Barrier certificates and closure certificates are used to verify safety[24], persistence[19], and LTL properties[19], respectively. We will provide their definitions and related conclusions.

Definition 2.1 (barrier certificate). Given a system $\mathfrak{S} = (\mathcal{X}, \mathcal{X}_0, f)$ and $\mathcal{X}_u \subseteq \mathcal{X}$, a bounded function $B(x) : \mathcal{X} \to \mathbb{R}$ is a *barrier certificate* with respect to $\mathcal{X}_u$ such that $x_0 \in \mathcal{X}_0, x_u \in \mathcal{X}_u, x \in \mathcal{X}$ and $x' = f(x)$. We have:

$$B(x_0) \geq 0, \tag{7}$$

$$B(x_u) < 0 \tag{8}$$

$$B(x) \geq 0 \Rightarrow B(x') \geq 0. \tag{9}$$

Theorem 2.2 (Non-Negativity). *If a barrier certificate $B(x)$ can be found, then it can be guaranteed that $B(x_i) \geq 0$ for any trajectory $\tau = \langle x_0, x_1, \dots \rangle \in \mathfrak{T}(\mathfrak{S})$.*

Corollary 2.3 (safety). *If a barrier certificate $B(x)$ can be found such that it ensures all* trajectories *in $\mathfrak{T}(\mathfrak{S})$ can not visit $\mathcal{X}_u$*

Definition 2.4 (closure certificate for persistence). Given a system $\mathfrak{S} = (\mathcal{X}, \mathcal{X}_0, f)$ and $\mathcal{X}_{VF} \subseteq \mathcal{X}$, a bounded function $T : \mathcal{X} \times \mathcal{X} \to \mathbb{R}$ is a closure certificate if there exists a value $\epsilon \in \mathbb{R}_{>0}$ such that for all states $x, y \in \mathcal{X}, x' = f(x), x_0 \in \mathcal{X}_0, y', y'' \in \mathcal{X}_{VF}$. We have:

$$T(x, x') \geq 0, \tag{10}$$

$$T(x', y) \geq 0 \Rightarrow T(x, y) \geq 0, \tag{11}$$

$$T(x_0, y') \geq 0 \wedge T(y', y'') \Rightarrow T(x_0, y'') \leq T(x_0, y') - \epsilon. \tag{12}$$

Lemma 2.5 (Non-Negativity). *Given a system $\mathfrak{S} = (\mathcal{X}, \mathcal{X}_0, f)$ and $\mathcal{X}_{VF} \subseteq \mathcal{X}$, if a persistence closure certificate T can be found, then it can be guaranteed that $T(x_i, x_j) \geq 0$, where $i < j$ and $\langle x_0, x_1, \dots \rangle \in \mathfrak{T}(\mathfrak{S})$.*

Theorem 2.6 (persistence). *Given a system $\mathfrak{S} = (\mathcal{X}, \mathcal{X}_0, f)$ and $\mathcal{X}_{VF} \subseteq \mathcal{X}$, if a persistence closure certificate T can be found, then it can be guaranteed that all* trajectories *of $\mathfrak{S}$ visit $\mathcal{X}_{VF}$ finitely often.*

Definition 2.7 (closure certificate for LTL). Consider a system $\mathfrak{S} = (\mathcal{X}, \mathcal{X}_0, f)$, a labelling function $\mathfrak{L} : \mathcal{X} \to \Sigma$ and a state-based NBA $\mathcal{A} = (\Sigma, Q, q_0, \delta, Acc)$ representing the negation of an LTL formula ϕ. A bounded function $T : \mathcal{X} \times Q \times \mathcal{X} \times Q \to \mathbb{R}$ is an closure certificate for ϕ if there exists a value $\epsilon \in \mathbb{R}_{>0}$ such that for all states $x, y, y' \in \mathcal{X}$, $x' = f(x)$, states $q_i, q_j \in Q$, and $q_i' \in \delta(q_i, \mathfrak{L}(x))$, we have:

$$T((x, q_i), (x', q_i')) \geq 0 \tag{13}$$

$$T((x', q_i'), (y, q_j)) \geq 0 \Rightarrow T((x, q_i), (y, q_j)) \geq 0 \tag{14}$$

and for all states $x_0 \in \mathcal{X}_0$, and $\ell, \ell' \in Acc$, we have:

$$\begin{aligned} &T((x_0, q_0), (y, \ell)) \geq 0 \wedge T((y, \ell), (y', \ell')) \geq 0 \\ &\Rightarrow T((x_0, q_0), (y', \ell')) \leq T((x_0, q_0), (y, \ell)) - \epsilon. \end{aligned} \tag{15}$$

Theorem 2.8 (LTL). *Consider a system $\mathfrak{S} = (\mathcal{X}, \mathcal{X}_0, f)$, a set of atomic propositions AP, a labelling function $\mathfrak{L} : \mathcal{X} \to 2^{AP}$ and an LTL formula ϕ. Let a state-based NBA $\mathcal{A} = (2^{AP}, Q, q_0, Acc_{state})$ represent $\neg\phi$. The existence of a closure certificate satisfying conditions (13)-(15) implies that $\mathfrak{S} \models_{\mathfrak{L}} \phi$.*

2.5 Problem Statement

How do we prove ω-regular properties ϕ, i.e., $\mathfrak{S} \models_{\mathfrak{L}} \phi$ and what advantages does this method offer compared to the state-triplet method and closure certificates?

3 Automata-based Methods

In this section, we present proof certificates and automata-based methods for the verification of ω-regular properties.

3.1 State Persistence Certificate

Definition 3.1 (state persistence certificate). Consider a system $\mathfrak{S} = (\mathcal{X}, \mathcal{X}_0, f)$.A bounded function $B_{sp}(x) : \mathcal{X} \to \mathbb{R}_{\geq 0}$ is a *state persistence certificate* for $\mathfrak{S}$ with respect to $\mathcal{X}_{VF} \subseteq \mathcal{X}$ if there exists a value $\epsilon > 0$ such that for $x \in \mathcal{X}, x_0 \in \mathcal{X}_0, x' = f(x)$, we have:

$$B_{sp}(x_0) \geq 0, \tag{16}$$

$$B_{sp}(x) \geq 0 \Rightarrow B_{sp}(x') \geq 0, \tag{17}$$

$$B_{sp}(x) \geq B_{sp}(x') + I_{\mathcal{X}_{VF}}(x')\epsilon, \tag{18}$$

$$B_{sp}(x) \geq B_{sp}(x') + I_{\mathcal{X}_{VF}}(x)\epsilon. \tag{19}$$

We have two types of *the state persistence certificate* based on the different conditions, but both types can be used to prove persistence. If B_{sp} satisfies (16, 17, 18), then B_{sp} is a type 1 *state persistence certificate*. If B_{sp} satisfies (16, 17, 19), then B_{sp} is a type 2 *state persistence certificate*.

Theorem 3.2 (persistence). *Given a system $\mathfrak{S} = (\mathcal{X}, \mathcal{X}_0, f)$ and $\mathcal{X}_{VF} \subseteq \mathcal{X}$, if a* type 1 *state persistence certificate B_{sp} with respect to $\mathcal{X}_{VF}$ can be found, then it can be guaranteed that τ visits(2.1) $\mathcal{X}_{VF}$ finitely often for any* trajectory $\tau = \langle x_0, x_1, \dots \rangle \in \mathfrak{T}(\mathfrak{S})$.

Proof. We assume there exists a trajectory τ that visits $\mathcal{X}_{VF}$ infinitely often. We extract the states in τ that visit $\mathcal{X}_{VF}$ in order to construct $\tau' = \langle y_0, y_1, \dots, y_n, \dots \rangle$, where $y_i \in \mathcal{X}_{VF}$. We now prove that $B_{sp}(y_i) \leq B_{sp}(x_0) - i\epsilon$ for all $i \in \mathbb{N}$. We proceed by induction on i.

- **Base case:** When $i = 0$, we have $B_{sp}(y_0) \leq B_{sp}(x_0)$ according to the Lemma (A.2).
- **Inductive step:** We assume the statement holds for $i = k$, i.e., $B_{sp}(y_k) \leq B_{sp}(x_0) - k\epsilon$. When $i = k + 1$:
  - If $y_{k+1} = f(y_k)$, we have $B_{sp}(y_{k+1}) \leq B_{sp}(y_k) - \epsilon$ according to the property (18). Combining this with the inductive hypothesis, we obtain $B_{sp}(y_{k+1}) \leq B_{sp}(x_0) - (k+1)\epsilon$.
  - If $y_{k+1} \neq f(y_k)$, i.e., y_k is not the *predecessor*(2.2) of y_{k+1}, so y_{k+1}^{pre} must appear after y_k in τ. So we have $B_{sp}(y_{k+1}^{pre}) \leq B_{sp}(y_k)$ according to the Lemma (A.3). We also have $B_{sp}(y_{k+1}^{pre}) \geq B_{sp}(y_{k+1}) + \epsilon$ according to the property (18). Combining these, we obtain $B_{sp}(y_{k+1}) + \epsilon \leq B_{sp}(y_k)$. Using the inductive hypothesis, we have $B_{sp}(y_{k+1}) \leq B_{sp}(x_0) - (k+1)\epsilon$.

Consequently, there exists $n \in \mathbb{N}$ such that $B_{sp}(y_n) \leq B_{sp}(x_0) - n\epsilon < 0$. This contradicts the Lemma (A.1). □

We have shown that type 1 *state persistence certificate* is sufficient to guarantee *persistence*(2.3). By using a similar proof method, we can easily prove that the type 2 *state persistence certificate* is able to achieve the same conclusion. That is, both types of state persistence certificates can ensure *persistence*. Next, we will demonstrate

that the *state persistence certificate* possesses stronger expressive power compared with the closure certificate for persistence.

Theorem 3.3 (EXPRESSIVENESS). *Given a system $\mathfrak{S} = (\mathcal{X}, \mathcal{X}_0, f)$, a set of states $\mathcal{X}_{VF}$ and a persistence closure certificate $T : \mathcal{X} \times \mathcal{X} \to \mathbb{R}$, one can compute a* state persistence certificate B_{sp}.

Proof. Since T (2.4) is a bounded function, let its upper bound be denoted as $T(x^*, y^*)$. We choose ϵ to be the same as the ϵ in T. Assume τ is an arbitrary *trajectory*, where $\tau = \langle x_0, x_1, \ldots, x_n, \ldots \rangle \in \mathfrak{T}(\mathfrak{S}), x_0 \in \mathcal{X}_0$, we define:

$$B_{sp}(x_0) = T(x^*, y^*) + \epsilon, \tag{20}$$

$$B_{sp}(x_{i+1}) = \begin{cases} B_{sp}(x_i) & \text{if } x_{i+1} \notin \mathcal{X}_{VF} \\ T(x_0, x_{i+1}) & \text{if } x_{i+1} \in \mathcal{X}_{VF} \end{cases}. \tag{21}$$

We now prove that $B_{sp}(x)$ is a type 1 *state persistence certificate* with respect to $\mathcal{X}_{VF}$.

- **Verification of (16):** We have $B_{sp}(x_0) = T(x^*, y^*) + \epsilon \geq T(x_0, x_0) + \epsilon > 0$ according to the Lemma (2.5).
- **Verification of (17):** We proceed by induction on i.
  - **Base case:** When $i = 1$, we have $B_{sp}(x_0) \geq 0$.
    - If $x_1 \notin \mathcal{X}_{VF}$, $B_{sp}(x_1) = B_{sp}(x_0) \geq 0$.
    - If $x_1 \in \mathcal{X}_{VF}$, we have $B_{sp}(x_1) = T(x_0, x_1) \geq 0$ according to the Lemma (2.5).
  - **Inductive Step:** We assume the statement holds for $i = k$ holds, i.e., $B_{sp}(x_{k-1}) \geq 0 \implies B_{sp}(x_k) \geq 0$. When $i = k + 1$ and $B_{sp}(x_k) \geq 0$:
    - If $x_{k+1} \notin \mathcal{X}_{VF}$, $B_{sp}(x_{k+1}) = B_{sp}(x_k) \geq 0$.
    - If $x_{k+1} \in \mathcal{X}_{VF}$, $B_{sp}(x_{k+1}) = T(x_0, x_{k+1}) \geq 0$ according to the Lemma (2.5).
- **Verification of (18):** We proceed by induction on i:
  - **Base case:** When $i = 0$, we have $B_{sp}(x_0) = T(x^*, y^*) + \epsilon$.
    - If $x_1 \in \mathcal{X}_{VF}$, then $B_{sp}(x_1) = T(x_0, x_1) \leq T(x^*, y^*)$. Thus, $B_{sp}(x_1) + \epsilon \leq B_{sp}(x_0)$.
    - If $x_1 \notin \mathcal{X}_{VF}$, then $B_{sp}(x_1) = B_{sp}(x_0)$.
  - **Inductive step:** Assume the statement holds for $i = k$, i.e., $B_{sp}(x_k) \geq B_{sp}(x_{k+1}) + I_{\mathcal{X}_{VF}}(x_{k+1})\epsilon$. When $i = k + 1$:
    - If $x_{k+2} \notin \mathcal{X}_{VF}$, then $B_{sp}(x_{k+2}) = B_{sp}(x_{k+1})$.
    - If $x_{k+2} \in \mathcal{X}_{VF}$, to prove $B_{sp}(x_{k+2}) + \epsilon \leq B_{sp}(x_{k+1})$, we first prove that for $n > m \geq 0$ and $x_n \in \mathcal{X}_{VF}$, $B_{sp}(x_m) - B_{sp}(x_n) \geq \epsilon$. We proceed by induction on m:
      - **Base case:** When $m = 0$, we have $B_{sp}(x_0) - B_{sp}(x_n) = T(x^*, y^*) + \epsilon - T(x_0, x_n) \geq \epsilon$.
      - **Inductive step:** we assume the statement holds for $m = l < n - 1$, i.e., $B_{sp}(x_l) - B_{sp}(x_n) \geq \epsilon$. When $m = l + 1 < n$, if $x_{l+1} \notin \mathcal{X}_{VF}$, we have $B_{sp}(x_{l+1}) = B_{sp}(x_l) \geq B_{sp}(x_n) + \epsilon$ according to the inductive hypothesis. If $x_{l+1} \in \mathcal{X}_{VF}$, we have $B_{sp}(x_{l+1}) = T(x_0, x_{l+1})$ and $B_{sp}(x_n) = T(x_0, x_n)$. We also have $T(x_{l+1}, x_n) \geq 0$ and $T(x_0, x_{l+1}) \geq 0$ according to the Lemma (2.5). Thus, we obtain $T(x_0, x_n) \leq T(x_0, x_{l+1}) - \epsilon$ according to the property (12), which implies $B_{sp}(x_{l+1}) - B_{sp}(x_n) \geq \epsilon$.

      Using this result, we conclude $B_{sp}(x_{k+2}) + \epsilon \leq B_{sp}(x_{k+1})$.

□

The theorem(3.3) asserts that if one can obtain the closure certificate for persistence, then one can construct a corresponding state persistence certificate to guarantee persistence.

3.2 State Safety Certificate

Definition 3.4 (state safety certificate). Given a system $\mathfrak{S} = (\mathcal{X}, \mathcal{X}_0, f)$, a labelling function $\mathfrak{L} : \mathcal{X} \to \Sigma$ and a Θ-based NBA $\mathcal{A} = (\Sigma, Q, q_0, \delta, Acc_\Theta)$ where $\Theta \in \{transition, state\}$. Construct the product system $\mathfrak{S} \times \mathcal{A} =$

$(\mathcal{Y}, \mathcal{Y}_0, F)$, where $\mathcal{Y} = \{(x,q) \mid x \in \mathcal{X}, q \in Q\}$, $\mathcal{Y}_0 = \{(x_0,q_0) \mid x_0 \in \mathcal{X}_0\}$, and $F(x,q) = \{(x',q') \mid x' = f(x) \wedge (q, \mathfrak{L}(x), q') \in \delta\}$. A bounded function $B_k^\Theta(x,q) : \mathcal{X} \times Q \to \mathbb{R}$ is a *state safety certificate* with respect to q_k, where $(x,q) \in \mathcal{Y}, (x_0,q_0) \in \mathcal{Y}_0, q_k \in Q, \mathcal{Y}_u = \{(x,q_k) \mid x \in \mathcal{X}\}, (x_0,q_0) \in \mathcal{Y}_0, (x_u,q_u) \in \mathcal{Y}_u, (x',q') \in F(x,q)$, we have:

$$B_k^\Theta(x_0,q_0) \geq 0, \tag{22}$$

$$B_k^\Theta(x_u,q_u) < 0, \tag{23}$$

$$B_k^\Theta(x,q) \geq 0 \Rightarrow B_k^\Theta(x',q') \geq 0. \tag{24}$$

3.3 Transition Persistence Certificate

Definition 3.5 (transition persistence certificate). Consider a system $\mathfrak{S} = (\mathcal{X}, \mathcal{X}_0, f)$, a labelling function $\mathfrak{L} : \mathcal{X} \to \Sigma$ and a Θ-based NBA $\mathcal{A} = (\Sigma, Q, q_0, \delta, Acc_\Theta)$, we define $\Delta_{k,l} = \{(q_k, \sigma, q_l) \mid \mathcal{P}(q_k,\sigma,q_l) \wedge (q_k,\sigma,q_l) \in \delta\}$ as the set of transitions that satisfy the predicate $\mathcal{P}$ and move from q_k to q_l. A bounded function $B_{\Delta_{k,l}}^\Theta(x) : \mathcal{X} \to \mathbb{R}$ is a *transition persistence certificate* with respect to $\Delta_{k,l}$ where $x_0 \in \mathcal{X}_0, x \in \mathcal{X}, x' = f(x)$, we have:

$$B_{\Delta_{k,l}}^\Theta(x_0) \geq 0, \tag{25}$$

$$B_{\Delta_{k,l}}^\Theta(x) \geq 0 \Rightarrow B_{\Delta_{k,l}}^\Theta(x') \geq 0, \tag{26}$$

$$B_{\Delta_{k,l}}^\Theta(x) \geq B_{\Delta_{k,l}}^\Theta(x') + I_{\Delta_{k,l}}(q_k, \mathfrak{L}(x), q_l)\epsilon. \tag{27}$$

Theorem 3.6 (ω-REGULAR PROPERTY)). *Given a system $\mathfrak{S} = (\mathcal{X}, \mathcal{X}_0, f)$, a labelling function $\mathfrak{L} : \mathcal{X} \to \Sigma$ and a state-based NBA $\neg\mathcal{A} = (\Sigma, Q, q_0, \delta, Acc_{state})$ representing the negation of an ω-regular property ϕ. For each accepting state $q_k \in Acc_{state}$, if we can find either:*

(1) *A corresponding state safety certificate, or*
(2) *Transition persistence certificates for the set $\Delta = \left\{(j,w,k) \in \delta \mid (j,k) \in Acc_{state}^{pair}\right\}$. (1)*

then it can be guaranteed that $\mathfrak{S} \models_\mathfrak{L} \phi$.

Proof. Given an accepting state $q_k \in \mathrm{Acc_{state}}$:

(1) If a state safety certificate B_k^{state} is found, by Lemma (A.4), any system trajectory will never visit q_k.
(2) Partition the transition set Δ to obtain $\Delta_{i,k} = \{(i,w,k) \in \Delta\}$. For each non-empty $\Delta_{i,k}$, find the corresponding transition persistence certificate $B_{\Delta_{i,k}}^{\mathrm{state}}$. By Lemma (A.5), any system trajectory will only finitely often visit these transitions leading to accepting state q_k.

This ensures that each accepting state q_k is visited only finitely many times. If such certificates exist for all accepting states in $\mathrm{Acc_{state}}$, then the system language is rejected by automaton $\neg\mathcal{A}$, implying $\mathcal{L}(\mathfrak{S}) \subseteq \mathcal{L}(\mathcal{A}_\phi)$, where $\mathcal{A}_\phi$ denotes the NBA corresponding to the ω-regular property ϕ. Consequently, we have $\mathfrak{S} \models_\mathfrak{L} \phi$. □

Theorem (3.6) illustrates the verification of ω-regular properties by using state safety certificates and transition persistence certificates. By constructing the product system $\mathfrak{S} \times \mathcal{A}$ to synchronize the states of system $\mathfrak{S}$ and automaton $\mathcal{A}$. The core idea of state safety certificate B_k^{state} is that we establish a barrier certificate B_k^{state} over this product system and block the automaton $\neg\mathcal{A}$ from reaching the accepting state q_k. The transition persistence certificate $B_{k,l}^{state}$ demonstrates that transitions to the accepting state q_l are visited only finitely many times for any trajectory.

Both state safety certificates and transition persistence certificates can be utilized to prove ω-regular properties. To illustrate the necessity of the two types of certificates, consider the following scenario where the synthesis of a transition persistence certificate is not feasible. For the sake of readability, subsequent property descriptions will be presented using LTL formulas. Assume the system indeed satisfies the LTL specification $G\neg a$, but the transition persistence certificate cannot be synthesized. To prove the property, we firstly construct an transition-based

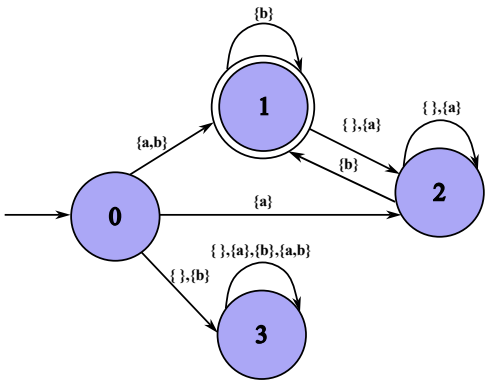

Fig. 1. This is a state-based NBA that represents the LTL formula $a \wedge G\,F\,b$

NBA $\mathcal{A}$ for Fa. Note that in the Figure 2, the alphabet is $\{\{\}, \{a\}\}$, the accepting transitions are $(q_0, \{\}, q_0)$ and $(q_0, \{a\}, q_0)$, so any letter in alphabet as input at q_0 only transition to q_0. According to the properties (25, 26, 27) of transition persistence certificate, $B_{0,0}(x_0) \geq 0$, $B_{0,0}(x) \geq 0 \Rightarrow B_{0,0}(x') \geq 0$, and $B_{0,0}(x) \geq B_{0,0}(x') + \epsilon$ should be satisfied. In particular, the third condition, which can lead to an infinite descent of the certificate, ultimately resulting in a contradiction with the Lemmas (A.1, A.2) and making it impossible to synthesize the certificate. This problem is caused by the reachability because if the *start state* q_0 of the accepting transtions can not be reached by all runs. However, this problem can be solved using the state safety certificate. By determining whether the state q_0 can be reached from state q_1, if a state safety certificate can be synthesized, it guarantees that q_0 is never reached, implying that it is finitely visited. If there exists a *trajectory* that allows $\mathcal{A}$ to visit q_0, then the state safety certificate cannot be synthesized. In such cases, the transition persistence certificate can serve as a witness that the relevant accepting transitions are finitely visited. Therefore, the combination of these two can complement each other's limitations.

In addition, we note that state safety certificates and transition persistence certificates can be applied to NBA whether they are *transition-based* or *state-based*. However, the question arises as to why *transition-based* NBAs are specifically proposed in this paper. For LTL property verification, the efficiency can be further improved using transition-based NBA. Both state safety certificates and transition persistence certificates can be utilized to prove LTL properties. Consider Figure 1, which depicts a *state-based* NBA $\mathcal{A}_1$ used to describe the LTL formula $a \wedge GFb$. In contrast, Figure 3 illustrates a *transition-based* NBA $\mathcal{A}_2$ to represent $a \wedge GFb$. We use p and q to represent the states of $\mathcal{A}_1$ and $\mathcal{A}_2$, respectively. Both figures represent the same formula. We observe that $\mathcal{A}_1$ has more states. When utilizing the state safety certificate, $\mathcal{A}_1$ seeks to cut off the simple paths from state p_0 to the accepting state p_1, while $\mathcal{A}_2$ aims to block the simple paths to the *start state* q_1 of the accepting transition $(q_1, \{b\}, q_1)$ from q_0. Regarding the transition persistence certificate, *transition-based* NBA $\mathcal{A}_2$ only needs to block the transition $(q_1, \{b\}, q_1)$, whereas a *state-based* NBA must block three transitions to accepting states: $(p_0, \{a, b\}, p_1)$, $(p_1, \{b\}, p_1)$, and $(p_2, \{b\}, p_1)$. That means we need to synthesize more certificates. Therefore, when verifying LTL properties, we would like to employ *transition-based* NBAs because they yield higher efficiency.

4 Compared with Other Methods

In the context of this discussion, we assume that the NBA for an LTL specification ϕ has already been constructed. The translation from an LTL formula to an NBA involves an exponential complexity with respect to the size of the formula [30]. The complexity of constructing the complement of an NBA is known to be EXPTIME [27].

4.1 The state-triplet Approach

Barrier certificate (2.1) separates the safe region $\mathcal{X}_{safe}$ from the unsafe region $\mathcal{X}_u$, ensuring that the system state remains within $\mathcal{X}_{safe}$ at all times and thus preventing the system behavior from visiting $\mathcal{X}_u$. If consider $\mathcal{X}_{VF}$ as the unsafe set, finding the corresponding barrier certificate B can guarantee the persistence of the system with respect to $\mathcal{X}_{VF}$. The state-triplet approach uses *state-based* NBA $\mathcal{A} = (\Sigma, Q, q_0, \delta, Acc_{state})$ to represent the complement of the LTL specification ϕ. This approach leverages barrier certificates to cut off all simple paths from the initial state q_0 to any accepting state in Acc_{state}. If one can find all certificates for all simple paths, this method ensures that *state-based* runs of *words* in $\mathfrak{L}(\mathfrak{S})$ never visit any accepting state in Acc_{state}, consequently, $\mathfrak{S} \models_{\mathfrak{L}} \phi$.

The concept of reachability is consistent between the *state safety certificate* and the state-triplet approach, but the *state safety certificate* achieves this by leveraging barrier certificates on the product system $\mathfrak{S} \times \mathcal{A}$. While The state-triplet needs to block all simple paths leading to $q_k \in Acc_{state}$ from q_0. The number of certificates in the state-triplet method is related to the number of triplets, but the number of state safety certificates is related to $|Acc_{state}|$. In addition, by finding a state safety certificate, it can avoid the need to unroll the automaton and expand all simple paths and then block them individually. The *transition persistence certificate* proposed in this paper allow *state-based* runs of *words* in $\mathfrak{L}(\mathfrak{S})$ to visit accepting states but ensures that they visit Acc_{state} finitely often by enforcing a decrease in the certificate value.

The combination of state safety certificates and transition persistence certificates can alleviate the conservatism of the state-triplet method. While maintaining its original capability of proving that a system satisfies specifications by cutting off reachability, this combination can also verify systems where there exist *state-based* runs that can reach accepting states from q_0 and meet the specifications.

4.2 Closure Certificate

According to the definition of the LTL closure certificate (2.7), the certificate is defined on $\mathcal{X} \times Q \times \mathcal{X} \times Q$. The increase in state dimensionality leads to longer synthesis times in practice. Therefore, searching for an LTL closure certificate imposes a heavy computational burden. For our certificates, whether it is the transition persistence certificate defined on $\mathcal{X}$ or the state safety certificate defined on $\mathcal{X} \times Q$, the state space is smaller compared to that of the closure certificate. Additionally, the number of transition persistence certificates is related to $|Acc^{pair}_{transition}|$ or $|Acc^{pair}_{state}|$, and the number of state safety certificates is related to $|Acc_{state}|$ or $|Acc^{start}_{transition}|$. Thus, the search space for all certificates is at most $\mathcal{X} \times Q \times Q$. Subsequent case studies will demonstrate that for cases previously involving closure certificates [19], the time required to synthesize the state safety certificates and transition persistence certificates can be completed in seconds, resulting in a significant improvement in synthesis efficiency.

5 Synthesis

5.1 SMT-based Approach

Counterexample-guided Inductive Synthesis (CEGIS) [28] has widespread applications in the synthesis of barrier certificates. We consider adopting the CEGIS method to synthesize *state safety certificates* and *transition persistence certificates*. To find the state safety certificates defined in (3.4), given a user-defined set of monomial functions $F_{basis,k} = \{p_{k,1}(x, q), \ldots, p_{k,z_k}(x, q)\}$ and a *transition-based* NBA $\mathcal{A} = (\Sigma, Q, q_0, \delta, Acc_{transition})$, where $p_{k,i}$ is a monomial defined on $\mathcal{X} \times Q$ with a degree not exceeding D, we define the template function $B^t_{k,temp}(x, q) = \sum_{i=1}^{z_k} c_{k,i} p_{k,i}(x, q)$ with respect to $q_k \in Acc^{start}_{transition}$, where $c_{k,i} \in \mathbb{R}$ are the coefficients to be determined. The function value of $B^t_{k,temp}$ can only be determined when x, q and the coefficients $c_{k,i}$ are given. Our goal is to find the coefficients $c_{k,i}$ and ϵ that satisfy the definition in (3.1).

The main idea of this method is to find candidate certificates $B^t_{k,c}$ by sampling states and encoding the conditions (22, 23, 24) into SMT queries related to these states. We denote these SMT queries as a set SQ. There are two main steps: first, find a candidate certificate based on SQ; second, check its validity and saturate SQ. The SMT solver uses SQ to find a candidate certificate $B^t_{k,c}$. Then, $B^t_{k,c}$ is checked for its validity. If counterexamples are collected during the verification process, new SMT queries for these counterexamples are added into SQ, and we return to the first step to find a new candidate certificate $B^t_{k,c}$. If no counterexample is found, that means $B^t_{k,c}$ is a *valid state safety certificate*, as it satisfies all conditions (22, 23, 24). The search for candidate certificates $B^t_{k,c}$ continues until a valid certificate is found or the maximum number of iterations is exceeded. $x' = f(x)$ represents the state following x. In the first step, the goal is to determine the values of $c_{k,i}$ and get a candidate certificate $B^t_{k,c}$. We sample N_1, N_2, N_3 states from $\mathcal{X}_0 \times \{q_0\}, \mathcal{X} \times Q, \mathcal{X} \times Acc^{start}_{transition}$ to form S_1, S_2, S_3, which are encoded as SMT queries as follows:

$$\bigwedge_{x \in S_1} B^t_{k,temp}(x) \geq 0, \tag{28}$$

$$\bigwedge_{x \in S_2} B^t_{k,temp}(x) \geq 0 \Rightarrow B^t_{k,temp}(x') \geq 0, \tag{29}$$

$$\bigwedge_{x \in S_3} B^t_{k,temp}(x) < 0. \tag{30}$$

Assuming that there exists $c_{k,i}$ that can satisfy all constraints (28, 29, 30), we use $c_{k,i,m}$ to represent the corresponding model of $c_{k,i}$. $c_{k,i,m}$ are substituted into $B^t_{k,temp}(x)$ to obtain a candidate state safety certificate $B^t_{k,c}(x)$. The goal of the second step is to discover counterexamples in $\mathcal{X} \times Q$ that violate (22, 23, 24). We encode negations of (22, 23, 24) as SMT queries. If any of these conditions are violated, the corresponding counterexamples are added into S_i and add corresponding conditions (28, 29, 30) as SMT queries into SQ to further constrain $c_{k,i}$. The encoding is as follows:

$$x \in \mathcal{X}_0 \wedge B^t_{k,c}(x, q_0) < 0, \tag{31}$$

$$x \in \mathcal{X} \wedge q \in Q \wedge (q, \mathfrak{L}(x), q') \in \delta \wedge B^t_{k,c}(x, q) \geq 0 \wedge B^t_{k,c}(x, q') < 0, \tag{32}$$

$$x \in \mathcal{X}_u \wedge q \in Acc^{start}_{transition} \wedge B^t_{k,c}(x, q) \geq 0. \tag{33}$$

Next, we will synthesize transition persistence certificates in (3.5). Given a *transition-based* NBA $\mathcal{A} = (\Sigma, Q, q_0, \delta, Acc_{transition})$, and a set of monomial functions $F_{basis,k,l} = \{p_{k,l,1}(x), \ldots, p_{k,l,z_{k,l}}(x)\}$ where $(k, l) \in Acc^{pair}_{transition}$, and then provide the corresponding certificate template $B^t_{k,l,temp}(x) = \sum_{i=1}^{z_{k,l}} c_{k,l,i} p_{k,l,i}(x)$. Then encode the conditions (25, 26, 27) for $c_{k,l,i}$ and ϵ_t. $x' = f(x)$ represents the state following x. In the first step, the goal is to determine the values of $c_{k,l,i}$ and ϵ_t, and then get candidate certificate $B^t_{k,l,c}$. We sample N_1, N_2 state from $\mathcal{X}_0, \mathcal{X}$ to form S_1 and S_2, which are encoded as SMT queries as follows:

$$\bigwedge_{x \in S_1} B^t_{k,l,temp}(x) \geq 0, \tag{34}$$

$$\bigwedge_{x \in S_2} B^t_{k,l,temp}(x) \geq 0 \Rightarrow B^t_{k,l,temp}(x') \geq 0, \tag{35}$$

$$\bigwedge_{x \in S_2} (q_k, \mathfrak{L}(x), q_l) \notin Acc^{k,l}_{transition} \wedge B^t_{k,l,temp}(x) \geq B^t_{k,l,temp}(x'), \tag{36}$$

$$\bigwedge_{x \in S_2} (q_k, \mathfrak{L}(x), q_l) \in Acc^{k,l}_{transition} \wedge B^t_{k,l,temp}(x) \geq B^t_{k,l,temp}(x') + \epsilon_t. \tag{37}$$

If solver can find $c_{k,l,i}$ and ϵ_t that satisfy constraints (34, 35, 36, 37), then substitute the corresponding models $c_{k,l,i,m}$ into $B^t_{k,l,temp}$ to obtain the candidate certificate $B^t_{k,l,c}$. We encode negations of (25, 26, 27) as SMT queries and substitute ϵ_m into these queries. We have:

$$x \in \mathcal{X}_0 \wedge B^t_{k,l,c}(x) < 0, \tag{38}$$

$$x \in \mathcal{X} \wedge B^t_{k,l,c}(x) \geq 0 \wedge B^t_{k,l,c}(x') < 0, \tag{39}$$

$$x \in \mathcal{X} \wedge (q_k, \mathfrak{L}(x), q_l) \in Acc^{k,l}_{transition} \wedge B^t_{k,l,c}(x) < B^t_{k,l,c}(x') + \epsilon_m, \tag{40}$$

$$x \in \mathcal{X} \wedge (q_k, \mathfrak{L}(x), q_l) \notin Acc^{k,l}_{transition} \wedge B^t_{k,l,c}(x) < B^t_{k,l,c}(x'). \tag{41}$$

Typically, CEGIS does not guarantee termination on uncountable state spaces; however, algorithms leveraging δ-completeness, such as branch-and-prune [13, 17], can ensure termination of the decision process. These algorithms return that either ϕ is unsatisfiable or a model of ϕ_δ, where ϕ_δ is a weaker consequence of ϕ such that $\phi \Rightarrow \phi_\delta$. It can ensure that the model has one-sided δ-bounded errors.

5.2 SOS

A subset $\mathcal{A} \subseteq \mathbb{R}^n$ is *semi-algebraic* if there exists a polynomial functions $h : \mathbb{R}^n \to \mathbb{R}_{\geq 0}$ such that $\mathcal{A} = \{x \mid h(x) \geq 0\}$. Given a *transition-based* NBA $\mathcal{A} = (\Sigma, Q, q_0, \delta, \mathrm{Acc}_{\mathrm{transition}})$, we want to synthesize the transition persistence certificate $B^t_{k,l}$ where $(k,l) \in \mathrm{Acc}^{\mathrm{pair}}_{\mathrm{transition}}$ by SOS programming [21]. Consider the sets $\mathcal{X}, \mathcal{X}_0, \mathcal{X}^{k,l}_{\mathrm{transition}}, \mathcal{X} \backslash \mathcal{X}^{k,l}_{\mathrm{transition}}$ as semi-algebraic sets where $\mathcal{X}^{k,l}_{\mathrm{transition}} = \{x \mid (q_k, \mathfrak{L}(x), q_l) \in Acc^{k,l}_{transition}\}$, i.e., there exist $g_{\mathcal{X}}$, $g_{\mathcal{X}_0}$, $g_{\mathcal{X}^{k,l}_{\mathrm{transition}}}$, $g_{\mathcal{X} \backslash \mathcal{X}^{k,l}_{\mathrm{transition}}}$ such that $\mathcal{X} = \{x \mid g_{\mathcal{X}}(x) \geq 0\}$, $\mathcal{X}_0 = \{x \mid g_{\mathcal{X}_0}(x) \geq 0\}$, $\mathcal{X}^{k,l}_{\mathrm{transition}} = \{x \mid g_{\mathcal{X}^{k,l}_{\mathrm{transition}}}(x) \geq 0\}$, $\mathcal{X} \backslash \mathcal{X}^{k,l}_{\mathrm{transition}} = \{x \mid g_{\mathcal{X} \backslash \mathcal{X}^{k,l}_{\mathrm{transition}}}(x) \geq 0\}$. We define the following polynomials:

$$B^t_{k,l}(x) - \lambda_{\mathcal{X}_0} g_{\mathcal{X}_0}(x), \tag{42}$$

$$B^t_{k,l}(x') - \beta B^t_{k,l}(x) - \lambda_{\mathcal{X}} g_{\mathcal{X}}(x), \tag{43}$$

$$B^t_{k,l}(x) - B^t_{k,l}(x') - \epsilon - \lambda_{\mathcal{X}^{k,l}_{transition}} g_{\mathcal{X}^{k,l}_{transition}}(x), \tag{44}$$

$$B^t_{k,l}(x) - B^t_{k,l}(x') - \lambda_{\mathcal{X} \backslash \mathcal{X}^{k,l}_{transition}} g_{\mathcal{X} \backslash \mathcal{X}^{k,l}_{transition}}(x). \tag{45}$$

We encode (26) into (43) and (43) is a stronger condition compared to (26). The multipliers $\lambda_{\mathcal{X}_0}, \lambda_{\mathcal{X}}, \lambda_{\mathcal{X}^{k,l}_{transition}}$ *and* $\lambda_{\mathcal{X} \backslash \mathcal{X}^{k,l}_{transition}}$ are sum-of squares over the state variable x. We assign a positive real number to β. The certificate synthesis problem reduces to verifying that polynomials (42), (43), (44) and (45) are SOS.

Lemma 5.1. *Given a* transition-based *NBA* $\mathcal{A} = (\Sigma, Q, q_0, \delta, Acc_{transition})$*, assuming that:* $\mathcal{X}$*,* $\mathcal{X}_0$*,* $\mathcal{X}^{k,l}_{transition}$*, and* $\mathcal{X} \backslash \mathcal{X}^{k,l}_{transition}$ *are semi-algebraic sets, and there exists an SOS polynomial* $B^t_{k,l}(x)$ *satisfying conditions (42), (43), (44) and (45), then* $B^t_{k,l}$ *is a transition persistence certificate with respect to* $Acc^{k,l}_{transition}$ *(6).*

To determine whether the above expressions are SOS, there are tools like `SOSTOOLS` [26] and `SumOfSquares` [33]. The complexity of verifying the SOS property of the certificate is $O\left(\binom{n+d}{d}^2\right)$[21], where n represents the dimensionality of state space and $2d$ is the degree of the polynomial. The complexity of verifying LTL polynomial specifications using *transition persistence certificates* is $O\left(2^{2|\phi|} \times \binom{n+d}{d}^2\right)$, where $|\phi|$ denotes the size of the LTL formula (number of literals). The SOS method is a sufficient condition method, which may exclude valid certificates that satisfy the certificate conditions but are not in the form of sums of squares. Therefore, it leads to conservatism due to the stronger satisfaction conditions.

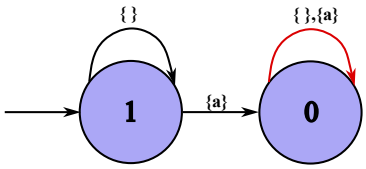

Fig. 2. This is a transition-based NBA that represents the LTL formula Fa and the red labelled transition is an accepting transition.

6 Case Study

We use Kuramoto oscillators [16] and a two-room temperature model [2] to demonstrate the process and efficiency of the proposed synthesis methods. In the first example, we consider the problem of safety verification. Here, we use LTL formula to represent safety and show that the safety of a one-dimensional and a two-dimensional Kuramoto oscillator can be verified by state safety certificates. In the second example, we consider the problem of persistence verification of a two-room temperature model and search for a transition persistence certificate. The reference machine is running Ubuntu 22.04.3 LTS (Intel i7-9750H with 8 GB of RAM).

6.1 Kuramoto Oscillator

The Kuramoto model has been used widely to describe chemical and biological osillators, with applications in neuroscience and modern power system analysis. As a first case study, we consider a system $\mathfrak{S} = (\mathcal{X}, \mathcal{X}_0, f)$ to model a Kuramoto oscillator whose dynamics are taken from [19], where $\mathcal{X} = [0, 2\pi]$ indicates the state set, $\mathcal{X}_0 = [\frac{4\pi}{9}, \frac{5\pi}{9}]$ the initial set of states, and $\mathcal{X}_u = [\frac{7\pi}{9}, \frac{8\pi}{9}]$ denotes the unsafe set of states. The state transition function f is defined as:

$$f(x) = x + t_s\Omega + t_sKsin(-x) - 0.532x^2 + 1.69,$$

where $x \in \mathcal{X}$ indicates the phase of the oscillator, t_s=0.1 is the sampling time, Ω=0.01 is the natural frequency, and $K = 0.0006$ is the coupling strength.

Firstly, we define $AP = \{a\}$, and use an LTL formula $G\neg a$ to describe the safety of $\mathfrak{S}$ with respect to $\mathcal{X}_u$, and we define labelling function:

$$\mathfrak{L}(x) = \begin{cases} \{a\} & \text{if } x \in \mathcal{X}_u \\ \{\} & \text{otherwise} \end{cases}$$

We use a *transition-based* NBA $\mathcal{A} = (2^{AP}, Q, q_1, \delta, Acc_{transition})$ in Figure 2 to represent the LTL formula Fa, the negation of the specification $G\neg a$. We want to find the state certificate $B_0^t(x, q)$ to block all accepting transitions $(0, \{\}, 0)$, $(0, \{a\}, 0)$ whose *start states* are all q_0. To do so, we use the CEGIS approach and define a template function as:

$$B^t_{0,temp}(x, q) = c_{0,0} + c_{0,1}x + c_{0,2}x^2 + c_{0,3}x^3 + c_{0,4}x^4 + c_{0,5}q + c_{0,6}q^2 + c_{0,7}q^3 + c_{0,8}xq$$

where $x \in \mathcal{X}$ and $q \in Q$. We sample 629 and 35 states from $\mathcal{X}$ and $\mathcal{X}_0$, we have $Q = \{0, 1\}$, $Acc^{start}_{transtion} = \{0\}$. According to (28, 29, 30), we encode corresponding z3 [11] queries to find candidate certificates. As z3 cannot handle the function $sin(-x)$, we instead use dReal [14] to find counterexamples according to (31, 32, 33) and set the precision of dReal to 0.0001. Finally, we find the *state safety certificate* $B_0^t(x, q) = -1 + 2q^2 - \frac{7}{16}xq$ that demonstrates the safety with respect to $\mathcal{X}_u$. The time taken for the CEGIS loop to terminate is around 4.25 seconds on the reference machine.

We consider the system $\mathfrak{S} = (\mathcal{X}, \mathcal{X}_0, f)$ to be a two-dimensional Kuramoto oscillator, where $\mathcal{X} = [0, \frac{8\pi}{9}] \times [0, \frac{8\pi}{9}]$ denotes the state set, $\mathcal{X}_0 = [0, \frac{\pi}{9}] \times [0, \frac{\pi}{9}]$ denotes the initial set of states, $\mathcal{X}_u = (\mathcal{X} \times [\frac{5\pi}{6}, \frac{8\pi}{9}]) \cup ([\frac{5\pi}{6}, \frac{8\pi}{9}] \times \mathcal{X})$ denotes the set of unsafe states and the transition function f is defined as:

$$f(x_1, x_2) = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} + \begin{bmatrix} t_s\Omega + 1.69 \\ t_s\Omega + 1.69 \end{bmatrix} + Kt_s \begin{bmatrix} \sin(x_2 - x_1) \\ \sin(x_1 - x_2) \end{bmatrix} - 0.532t_s \begin{bmatrix} x_1^2 \\ x_2^2 \end{bmatrix}$$

where $(x_1, x_2) \in \mathcal{X}$ indicates the phase of the oscillators, and the remaining constants have the same values as the one-dimensional case. we define labelling function:

$$\mathfrak{L}(x_1, x_2) = \begin{cases} \{a\} & \text{if } x_1 \in [\frac{5\pi}{6}, \frac{8\pi}{9}] \text{ or } x_2 \in [\frac{5\pi}{6}, \frac{8\pi}{9}] \\ \{\} & \text{otherwise} \end{cases}$$

We still use $\mathcal{A}$ to represent safety specification in Figure 2. We consider the template function of the *state safety certificate* to be:

$$B^t_{0,temp}(x_1, x_2, q) = c_0 + c_1x_1 + c_2x_2^2 + c_3x_1x_2 +$$

$$c_4x_1^3 + c_5q + c_6q^2 + c_7qx_1^2 + c_8x_2q$$

We then search for the *state safety certificate* via the CEGIS method. We find the *state safety certificate*:

$$B^t_0(x_1, x_2, q) = -1 + \frac{1}{8}x_1x_2 + \frac{3}{2}q^2 - \frac{3}{32}qx_1^2 - \frac{153}{512}x_2q$$

that demonstrates safety. The time taken for this CEGIS loop to terminate is around 7.27 seconds on the reference machine.

The time taken for the CEGIS loop to synthesize a persistence closure certificate is around 10 minutes for 1-dimension problem and synthesize a LTL closure certificate is around an hour and 50 minutes for 2-dimension problem on a machine running MacOS 11.2 (Intel i9-9980HK with 64 GB of RAM) [19].

6.2 Two Room Temperature Model

As a second case study, we consider our system $\mathfrak{S} = (\mathcal{X}, \mathcal{X}_0, f)$ to be an interconnected two-room temperature model, where $\mathcal{X} = [20, 34] \times [20, 34]$ indicates the temperature of these two rooms, $\mathcal{X}_0 = [21, 24] \times [21, 24]$ denotes the initial set of states, and the transition function is defined as:

$$f(x_1, x_2) = A \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} + \mu T_h \begin{bmatrix} u(x_1) \\ u(x_2) \end{bmatrix} + \theta \begin{bmatrix} T_e \\ T_e \end{bmatrix}$$

where x_i represents the temperature of room i, for all $i \in \{1, 2\}$, the matrix A is:

$$A = \begin{bmatrix} 1 - 2\alpha - \theta - \mu u(x_1) & \alpha \\ \alpha & 1 - 2\alpha - \theta - \mu u(x_2) \end{bmatrix}$$

where constants $\alpha = 0.004$, $\theta = 0.01$, and $\mu = 0.15$ represent the conduction factors. The function $u(x)$ denotes the temperature controller, which is defined as $u(x_i) = 0.59 - 0.011x_i$. The value $T_h = 40$ denotes the heater temperature in 40 degrees Celsius, and $T_e = 0$ represents the ambient temperature. In this setting, we hope that any *trajectory* of the system starting from $\mathcal{X}_0$ must visit the region $[20, 26] \times [20, 26]$ finitely often.

We consider the atomic propositions $AP = \{a, b\}$, and the alphabet $\Sigma = \{\{\}, \{a\}, \{b\}, \{a, b\}\}$. Here, we mark the states $(x_1, x_2) \in \mathcal{X}_0$ with the atomic proposition a. We mark a state $(x_1, x_2) \in \mathcal{X}$ with atomic proposition b if

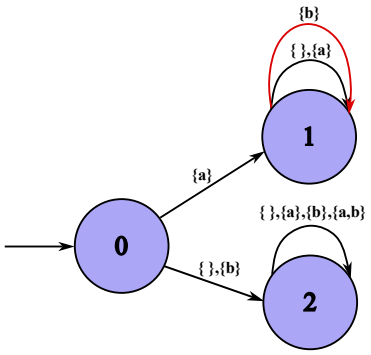

Fig. 3. This is a transition-based NBA that represents the LTL formula $a \wedge GFb$.

$(x_1, x_2) \in [20, 26] \times [20, 26]$. All other states are not marked with any atomic proposition. Observe that a state (x_1, x_2) may be marked with both atomic propositions a, and b, or neither. We define the labelling function as:

$$\mathfrak{L}(x_1, x_2) = \begin{cases} \{a, b\} & \text{if } (x_1, x_2) \text{ is marked with both } a, b \\ \{a\} & \text{if } (x_1, x_2) \text{ is marked with only } a \\ \{b\} & \text{if } (x_1, x_2) \text{ is marked with only } b \\ \{\} & \text{otherwise} \end{cases}$$

We use an LTL formula $a \Longrightarrow F\,G\,\neg b$ to describe the specification. The negation of the specification is $a \wedge GFb$ and a *transition-based* NBA $\mathcal{A} = (\Sigma, Q, q_0, \delta, Acc_{transition})$ in Figure 3 describes this formula. Firstly, we use the CEGIS method to synthesize the *state safety certificate* $B_1^t(x, q)$ to check the reachability from q_0 to q_1. The synthesis fails, taking 16.24 seconds on the reference machine. This proves that there may exist a system *trajectory* that makes the automaton $\mathcal{A}$ transition from q_0 to q_1. Therefore, we attempt to synthesize the *transition persistence certificate*.

We consider a template function of the *transition persistence certificate* specified as:

$$\begin{aligned} &B^t_{1,1,temp}(x_1, x_2) = c_0 + c_1 I_{\mathcal{X}_0}(x_1, x_2) + c_2 I_{\mathcal{X}_{VF}}(x_1, x_2) + c_3 x_1 I_{\mathcal{X}_0}(x_1, x_2) + \\ &c_4 x_2 I_{\mathcal{X}_0}(x_1, x_2) + c_5 x_1 I_{\mathcal{X}_{VF}}(x_1, x_2) + c_6 x_2 I_{\mathcal{X}_{VF}}(x_1, x_2) + c_7 max(x_1, x_2) + \\ &c_8 x_1^2 + c_9 x_2^2 \end{aligned}$$

where $(x_1, x_2) \in \mathcal{X}$. Firstly, we uniformly sample states from $\mathcal{X}$ and $\mathcal{X}_0$. Then we code corresponding (34, 35, 36, 37) z3 queries to find a candidate certificate and use z3 to find counterexamples according to these queries (38, 39, 40, 41). The resulting certificate:

$$B^t_{1,1}(x_1, x_2) = 27 I_{\mathcal{X}_{VF}}(x_1, x_2) - x_1 I_{\mathcal{X}_{VF}}(x_1, x_2)$$

and $\epsilon = 0.25$. The time taken to find the certificate is around 8.77 seconds on the reference machine.

In addition, we use the SOS method (42, 43, 44, 45) to synthesize the certificate where $\beta = 0.01$, the resulting SOS *transition persistence certificate*:

$$\begin{aligned} B_{1,1}(x_1, x_2) = &\ 3.158(-0.004x_1 - 0.016x_2 + 1)^2 + \\ &1.11(-0.999x_1 + x_2)^2 + 0.001x_1^2 \end{aligned}$$

and $\epsilon = 0.0006844$. The time taken to find the certificate is around 1.27 seconds on the reference machine.

The time taken for the CEGIS loop to synthesize LTL closure certificate is around an hour and 30 minutes for this problem on a machine running MacOS 11.2 (Intel i9-9980HK with 64 GB of RAM) [19].

7 Conclusion

This paper proposes proof certificates for ensuring that the system satisfies specifications. Combining state safety certificates and transition persistence certificates can play a complementary role in verification. We demonstrate that the proposed certificates can subsume the state-triplet method and have a smaller search space compared to the closure certificate. Finally, the efficiency of the synthesis has been demonstrated through case studies. As future work, we plan to investigate data-driven approaches to find these certificates, as well as investigate their use in synthesizing controllers.

References

[1] Alessandro Abate, Mirco Giacobbe, and Diptarko Roy. 2024. Stochastic omega-regular verification and control with supermartingales. In *International Conference on Computer Aided Verification*. Springer, 395–419.

[2] Mahathi Anand, Abolfazl Lavaei, and Majid Zamani. 2024. Compositional synthesis of control barrier certificates for networks of stochastic systems against ω-regular specifications. *Nonlinear Analysis: Hybrid Systems* 51 (2024), 101427.

[3] Matin Ansaripour, Krishnendu Chatterjee, Thomas A Henzinger, Mathias Lechner, and Đorđe Žikelić. 2023. Learning provably stabilizing neural controllers for discrete-time stochastic systems. In *International Symposium on Automated Technology for Verification and Analysis*. Springer, 357–379.

[4] Christel Baier and Joost-Pieter Katoen. 2008. *Principles of model checking*. MIT press.

[5] J Richard Büchi. 1966. Symposium on decision problems: On a decision method in restricted second order arithmetic. In *Studies in Logic and the Foundations of Mathematics*. Vol. 44. Elsevier, 1–11.

[6] Aleksandar Chakarov and Sriram Sankaranarayanan. 2013. Probabilistic program analysis with martingales. In *Computer Aided Verification: 25th International Conference, CAV 2013, Saint Petersburg, Russia, July 13-19, 2013. Proceedings 25*. Springer, 511–526.

[7] Krishnendu Chatterjee, Hongfei Fu, Petr Novotnỳ, and Rouzbeh Hasheminezhad. 2016. Algorithmic analysis of qualitative and quantitative termination problems for affine probabilistic programs. In *Proceedings of the 43rd Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*. 327–342.

[8] Krishnendu Chatterjee, Amir Goharshady, Ehsan Goharshady, Mehrdad Karrabi, and Đorđe Žikelić. 2024. Sound and complete witnesses for template-based verification of LTL properties on polynomial programs. In *International Symposium on Formal Methods*. Springer, 600–619.

[9] Krishnendu Chatterjee, Ehsan Kafshdar Goharshady, Petr Novotnỳ, and Đorđe Žikelić. 2024. Equivalence and similarity refutation for probabilistic programs. *Proceedings of the ACM on Programming Languages* 8, PLDI (2024), 2098–2122.

[10] Jean-Michel Couvreur. 1999. On-the-fly verification of linear temporal logic. In *International Symposium on Formal Methods*. Springer, 253–271.

[11] Leonardo De Moura and Nikolaj Bjørner. 2008. Z3: An efficient SMT solver. In *International conference on Tools and Algorithms for the Construction and Analysis of Systems*. Springer, 337–340.

[12] Alexandre Duret-Lutz, Alexandre Lewkowicz, Amaury Fauchille, Thibaud Michaud, Étienne Renault, and Laurent Xu. 2016. Spot 2.0 — A Framework for LTL and omega-Automata Manipulation. In *Automated Technology for Verification and Analysis*, Cyrille Artho, Axel Legay, and Doron Peled (Eds.). Springer International Publishing, Cham, 122–129.

[13] Sicun Gao, Jeremy Avigad, and Edmund M Clarke. 2012. δ-complete decision procedures for satisfiability over the reals. In *International Joint Conference on Automated Reasoning*. Springer, 286–300.

[14] Sicun Gao, Soonho Kong, and Edmund M Clarke. 2013. dReal: An SMT solver for nonlinear theories over the reals. In *International conference on automated deduction*. Springer, 208–214.

[15] Dimitra Giannakopoulou and Flavio Lerda. 2002. From states to transitions: Improving translation of LTL formulae to Büchi automata. In *International Conference on Formal Techniques for Networked and Distributed Systems*. Springer, 308–326.

[16] Yufeng Guo, Dongrui Zhang, Zhuchun Li, Qi Wang, and Daren Yu. 2021. Overviews on the applications of the Kuramoto model in modern power system analysis. *International Journal of Electrical Power & Energy Systems* 129 (2021), 106804.

[17] Soonho Kong, Armando Solar-Lezama, and Sicun Gao. 2018. Delta-decision procedures for exists-forall problems over the reals. In *International Conference on Computer Aided Verification*. Springer, 219–235.

[18] Jan Křetínský, Tobias Meggendorfer, and Salomon Sickert. 2018. Owl: A Library for omega-Words, Automata, and LTL. In *Automated Technology for Verification and Analysis*, Shuvendu K. Lahiri and Chao Wang (Eds.). Springer International Publishing, Cham, 543–550.

[19] Vishnu Murali, Ashutosh Trivedi, and Majid Zamani. 2024. Closure certificates. In *Proceedings of the 27th ACM International Conference on Hybrid Systems: Computation and Control*. 1–11.
[20] Grigory Neustroev, Mirco Giacobbe, and Anna Lukina. 2025. Neural continuous-time supermartingale certificates. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 39. 27538–27546.
[21] Pablo A Parrilo. 2003. Semidefinite programming relaxations for semialgebraic problems. *Mathematical programming* 96 (2003), 293–320.
[22] Amir Pnueli. 1977. The temporal logic of programs. In *18th annual symposium on foundations of computer science (sfcs 1977)*. ieee, 46–57.
[23] Andreas Podelski and Andrey Rybalchenko. 2004. Transition invariants. In *Proceedings of the 19th Annual IEEE Symposium on Logic in Computer Science, 2004.* IEEE, 32–41.
[24] Stephen Prajna and Ali Jadbabaie. 2004. Safety verification of hybrid systems using barrier certificates. In *International workshop on hybrid systems: Computation and control*. Springer, 477–492.
[25] Stephen Prajna, Ali Jadbabaie, and George J Pappas. 2007. A framework for worst-case and stochastic safety verification using barrier certificates. *IEEE Trans. Automat. Control* 52, 8 (2007), 1415–1428.
[26] Stephen Prajna, Antonis Papachristodoulou, and Pablo A Parrilo. 2002. Introducing SOSTOOLS: A general purpose sum of squares programming solver. In *Proceedings of the 41st IEEE Conference on Decision and Control, 2002.*, Vol. 1. IEEE, 741–746.
[27] S. Safra. 1988. On the complexity of omega-automata. In *[Proceedings 1988] 29th Annual Symposium on Foundations of Computer Science*. 319–327. doi:10.1109/SFCS.1988.21948
[28] Armando Solar-Lezama. 2008. *Program synthesis by sketching*. University of California, Berkeley.
[29] Wolfgang Thomas. 1990. Automata on infinite objects. In *Formal Models and Semantics*. Elsevier, 133–191.
[30] Moshe Y Vardi. 2005. An automata-theoretic approach to linear temporal logic. *Logics for concurrency: structure versus automata* (2005), 238–266.
[31] Tichakorn Wongpiromsarn, Ufuk Topcu, and Andrew Lamperski. 2015. Automata theory meets barrier certificates: Temporal logic verification of nonlinear systems. *IEEE Trans. Automat. Control* 61, 11 (2015), 3344–3355.
[32] Bai Xue, Renjue Li, Naijun Zhan, and Martin Fränzle. 2021. Reach-avoid analysis for stochastic discrete-time systems. In *2021 American Control Conference (ACC)*. IEEE, 4879–4885.
[33] Chenyang Yuan. [n. d.]. *SumOfSquares.py*. https://github.com/yuanchenyang/SumOfSquares.py
[34] Đorđe Žikelić, Mathias Lechner, Thomas A Henzinger, and Krishnendu Chatterjee. 2023. Learning control policies for stochastic systems with reach-avoid guarantees. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 37. 11926–11935.
[35] Đorđe Žikelić, Mathias Lechner, Abhinav Verma, Krishnendu Chatterjee, and Thomas Henzinger. 2023. Compositional policy learning in stochastic control systems with formal guarantees. *Advances in Neural Information Processing Systems* 36 (2023), 47849–47873.

A Proof

Lemma A.1. *Given a system $\mathfrak{S} = (\mathcal{X}, \mathcal{X}_0, f)$, $\mathcal{X}_{VF} \subseteq \mathcal{X}$ and any* trajectory $\tau \in \mathfrak{T}(\mathfrak{S})$*, if a* type 1 *state persistence certificate B_{sp} with respect to $\mathcal{X}_{VF}$ can be found, then it can be guaranteed that $B_{sp}(x_i) \geq 0$, where $i \in \mathbb{N}$ and $\tau = \langle x_0, x_1, \dots \rangle$.*

Proof. We proceed by induction on i. For the base case, when $i = 0$, we have $B_{sp}(x_0) \geq 0$ according to the property (16). For the inductive step, we assume that the statement holds for $i = k$, i.e., $B_{sp}(x_k) \geq 0$. When $i = k + 1$, we have $B_{sp}(x_{k+1}) \geq 0$ according to the property (17). □

Lemma A.2. *Given a system $\mathfrak{S} = (\mathcal{X}, \mathcal{X}_0, f)$, $\mathcal{X}_{VF} \subseteq \mathcal{X}$ and any* trajectory $\tau = \langle x_0, x_1, \dots \rangle \in \mathfrak{T}(\mathfrak{S})$*, if a* type 1 *state persistence certificate B_{sp} with respect to $\mathcal{X}_{VF}$ can be found, then it can be guaranteed that $B_{sp}(x_i) \leq B_{sp}(x_0)$ for $i \in \mathbb{N}$.*

Proof. We proceed by induction on i. For the base case, when $i = 0$, we have $B_{sp}(x_0) \leq B_{sp}(x_0)$. For the inductive step, we assume that the statement holds for $i = k$, i.e., $B_{sp}(x_k) \leq B_{sp}(x_0)$. When $i = k + 1$, we have $B_{sp}(x_{k+1}) \leq B_{sp}(x_k)$ according to the property (18). Combining this with the inductive hypothesis, we have $B_{sp}(x_{k+1}) \leq B_{sp}(x_0)$. □

Lemma A.3. *Given a system $\mathfrak{S} = (\mathcal{X}, \mathcal{X}_0, f)$, $\mathcal{X}_{VF} \subseteq \mathcal{X}$ and any* trajectory $\tau = \langle x_0, x_1, \dots \rangle \in \mathfrak{T}(\mathfrak{S})$*, if a* type 1 *state persistence certificate B_{sp} can be found, then it can be guaranteed that $B_{sp}(x_i) \geq B_{sp}(x_j)$ for $i \in \mathbb{N}$, $j \in \mathbb{N}_{>i}$.*

Proof. We use double induction. We perform induction on i. For the base case, when $i = 0$, we have $B_{sp}(x_0) \geq B_{sp}(x_j)$ according to the Lemma (A.2). For the inductive step, we assume the statement holds for $i = k$, i.e.,

$B_{sp}(x_k) \geq B_{sp}(x_j)$ for $k < j-1$. When $i = k+1$, we proceed by induction on j. For the base case, When $j = k+2$, we have $B_{sp}(x_{k+1}) \geq B_{sp}(x_{k+2})$ according to the property (18). For the inductive step, we assume the statement holds for $j = k+1+l$,

$$B_{sp}(x_{k+1}) \geq B_{sp}(x_{k+1+l}). \tag{46}$$

When $j = k+l+2$, we have $B_{sp}(x_{k+l+1}) \geq B_{sp}(x_{k+l+2})$ according to the property (18). We have $B_{sp}(x_{k+1}) \geq B_{sp}(x_{k+l+2})$ according to the inductive hypothesis (46). □

Theorem A.4. *Given a system $\mathfrak{S} = (\mathcal{X}, \mathcal{X}_0, f)$, a trajectory $\tau = \langle x_0, x_1, \dots \rangle \in \mathfrak{T}(\mathfrak{S})$, a labelling function $\mathfrak{L} : \mathcal{X} \to \Sigma$ and a Θ-based NBA $\mathcal{A} = (\Sigma, Q, q_0, \delta, Acc_\Theta)$, if a τ-based state safety certificate $B_k^\Theta(x, q)$ with respect to $q_k \in Q$ can be found, then it can be guaranteed that all Θ-based runs of all* words *in $\mathfrak{L}(\mathfrak{S})$ can not visit the state q_k.*

Proof. According to Corollary (2.3), any *trajectory* of the system $\mathfrak{S} \times \mathcal{A}$ can not visit $\mathcal{Y}_u$. Therefore, Θ-based runs of all *words* in $\mathfrak{L}(\mathfrak{S})$ can not visit q_k. □

Theorem A.5. *Given a system $\mathfrak{S} = (\mathcal{X}, \mathcal{X}_0, f)$, a Θ-based NBA $\mathcal{A} = (\Sigma, Q, q_0, \delta, Acc_\Theta)$ and a set of transitions $\Delta_{k,l} \subseteq \delta$, if a transition persistence certificate $B^\Theta_{\Delta_{k,l}}$ can be found, then it can be guaranteed that all Θ-based runs of the words in $\mathfrak{L}(\mathfrak{S})$ make transitions in $\Delta_{k,l}$ happen finitely often.*

Proof. We define $\mathcal{X}_\Delta^{k,l} = \{x | (q_k, \mathfrak{L}(x), q_l) \in \Delta_{k,l}\}$. We prove that $B^\Theta_{\Delta_{k,l}}$ is a type 2 *state persistence certificate* with respect to $\mathcal{X}_\Delta^{k,l}$. The properties (16, 17) can be satisfied according to the properties (25, 26). We have $x \in \mathcal{X}_\Delta^{k,l}$ iff $(q_k, \mathfrak{L}(x), q_l) \in \Delta_{k,l}$ according to the the definition of $\mathcal{X}_\Delta^{k,l}$. Therefore, $I_{\mathcal{X}_\Delta^{k,l}}(x) = I_{\Delta_{k,l}}(q_k, \mathfrak{L}(x), q_l)$. we have $B^\Theta_{\Delta_{k,l}}(x) \geq B^\Theta_{\Delta_{k,l}}(x') + I_{\mathcal{X}_\Delta^{k,l}}(x)\epsilon$ according to the property (27). Therefore, it is guaranteed that any trajectory τ visits $\mathcal{X}_\Delta^{k,l}$ finitely often, which ensures that the Θ-based runs of the *word* $\mathfrak{L}(\tau)$ make transitions in $\Delta_{k,l}$ happen finitely often. □